Julia Transpiler

Szűcs Gergely

Julia AST

Symbol

A name in the code that can be used to refer to things based on the context of their evaluation (it represents something, but it has no actual binding and is fully context-independent).

:x -> refers to something named *x*

QuoteNode

Wraps Symbol values in the AST

LineNumberNode

Stores source code information

GlobalRef

Used internally by the Julia compiler

Literals

Expr

Used for every other language construct Julia supports.

Has a *head* field, which describes the type of expression it represents with a Symbol.

And an *args* field, which is a list of other AST nodes, its structure usually implied by *head*.

Julia AST

```julia
Expr(:call, :(+), :x, 2)
Expr(:ref, :arr, :i)
```

```julia
Expr(:call, :transpose,
    Expr(
        :call, :mat3,
        Expr(
            :(:), 1, 9
        )
    )
)
```

Expr

Used for every other language construct Julia supports.

Has a *head* field, which describes the type of expression it represents with a Symbol.

And an *args* field, which is a list of other AST nodes, its structure usually implied by *head*.

Transpiler Structure

Stages

Transforms the code from one structurally correct state to another one

Usually performs a single task

Ideally runs exactly one full traversal on the AST

Internally: functions or functions with output decorators

Pipelines

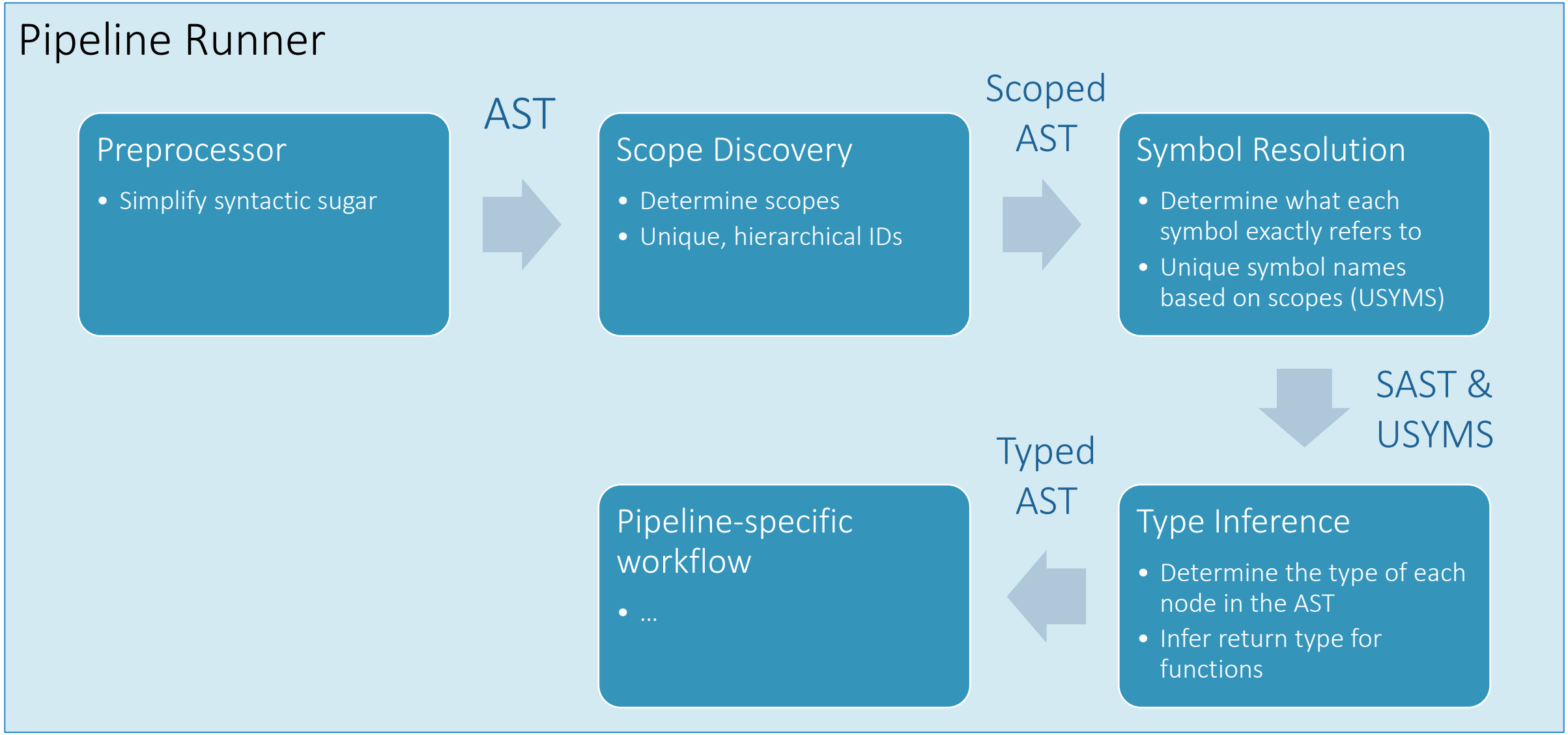
Transpiles a Julia function (or functions) to some target language

Chains a series of stages together

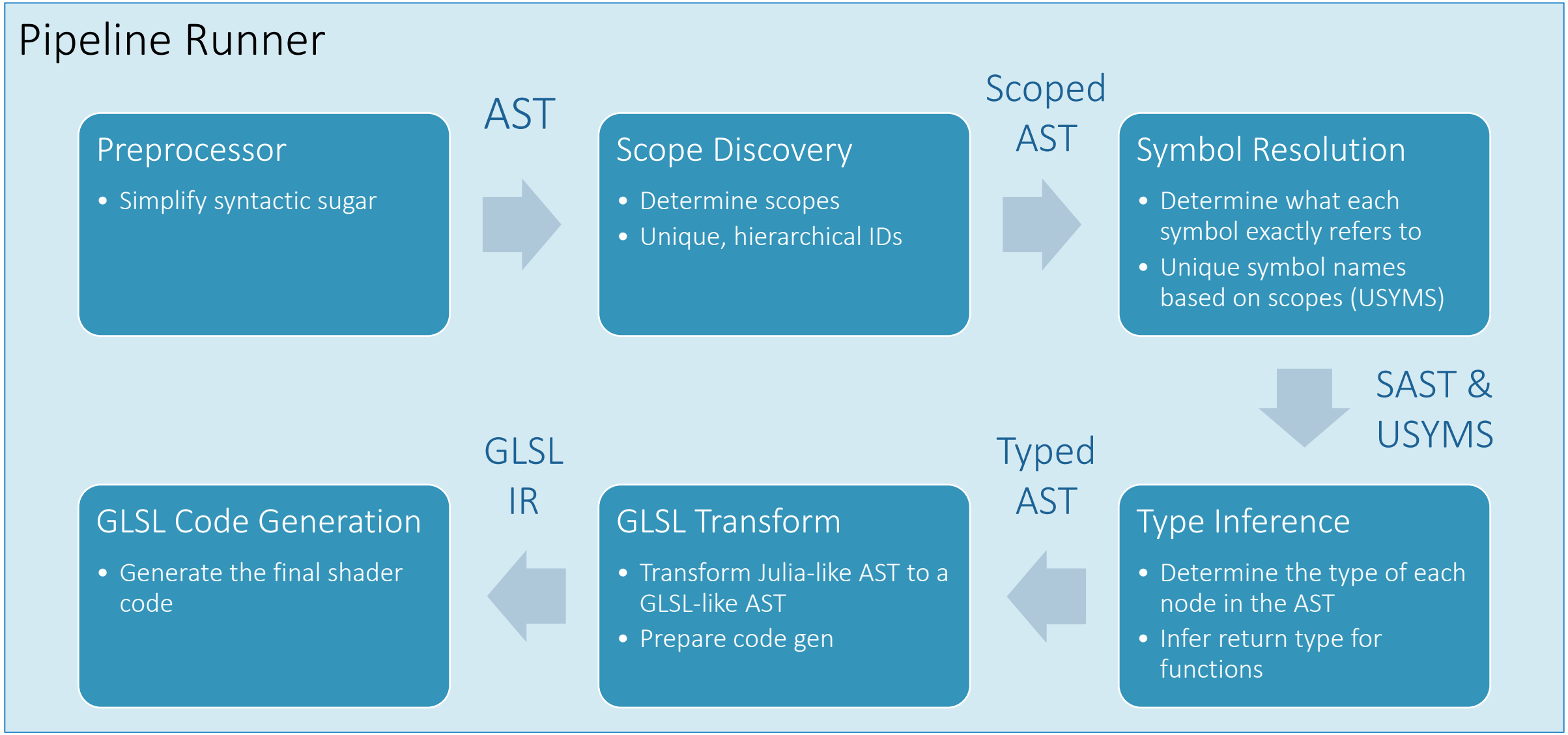
Provides context through abstract interfaces for stages

Internally: a list of Stages and a Context interface, which can handle the core pipeline's operations

Core Pipeline



GLSL Pipeline



Preprocessor

AST to AST transformation

Two tasks:

- remove easily transformable syntactic sugar (e.g. comparison chaining)
- ensure later stages can expect atomicity

Only performs equivalent transformations

By default, the input Julia function only gets defined after this stage (although this can be overridden by the pipeline)

Preprocessor

```
function preprocessor_test()
    x, y, z = 1, 2, 3

    if x < y < z
        x += y
    end
end
```

```
function preprocessor_test()
    x = 1
    y = 2
    z = 3

    if x < y && y < z
        x = x + y
    end
end
```

Scope Discovery

AST to Scoped AST (SAST) transformation

Traverses the tree from top to bottom (pre-order)

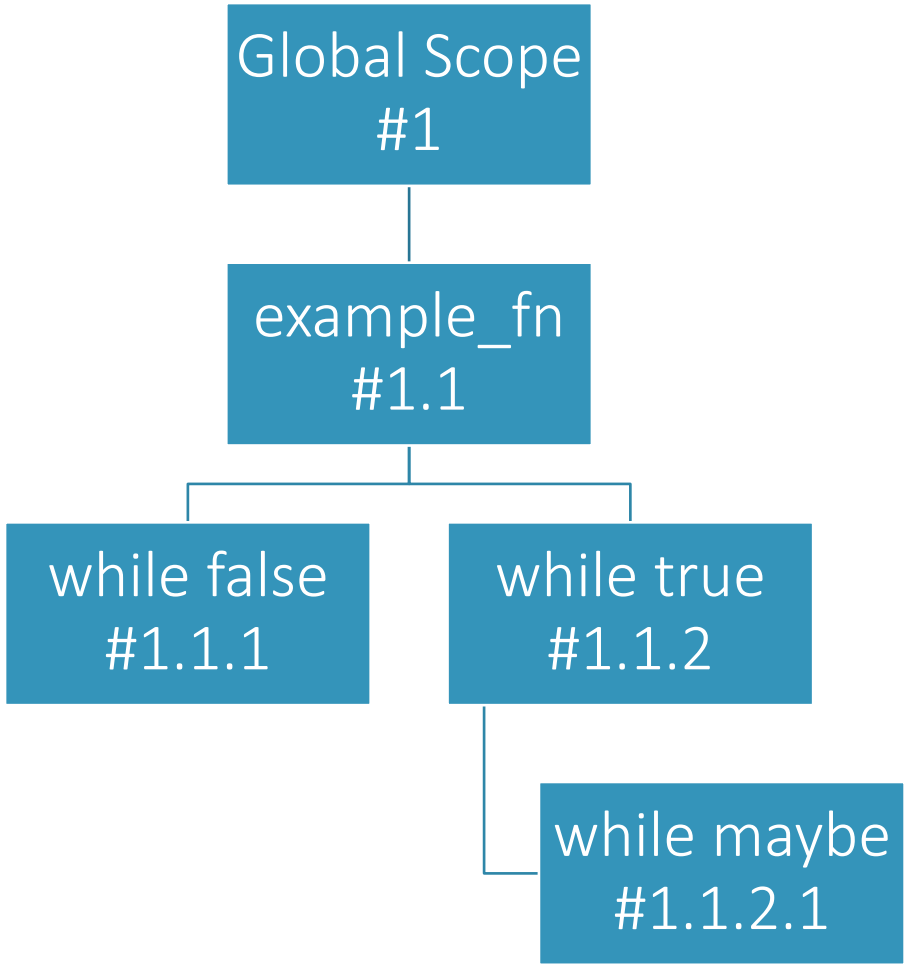
For every AST node that opens a new scope (loops, function defs, etc) it opens a new sub-scope with a unique ID

Respects Julia's hard/soft scope system and tags each scope accordingly

(even though it didn't really matter in the end)

Scope Discovery

```
# Scope 1

function example_fn()
    # Scope 1.1

    while false
        # Scope 1.1.1
    end

    while true
        # Scope 1.1.2

        while rand() < 0.5
            # Scope 1.1.2.1
        end
    end
end
```



Symbol Resolution

SAST to SAST transformation

Traverses the tree from top to bottom (pre-order)

In Julia, a symbol can only refer to one “thing” in a single scope

Renames each symbol to be unique inside the function

Checks symbol usage rules (e.g. access before initialization)

Outputs a list of unique symbols and mappings based on symbol-scope pairs along with the SAST

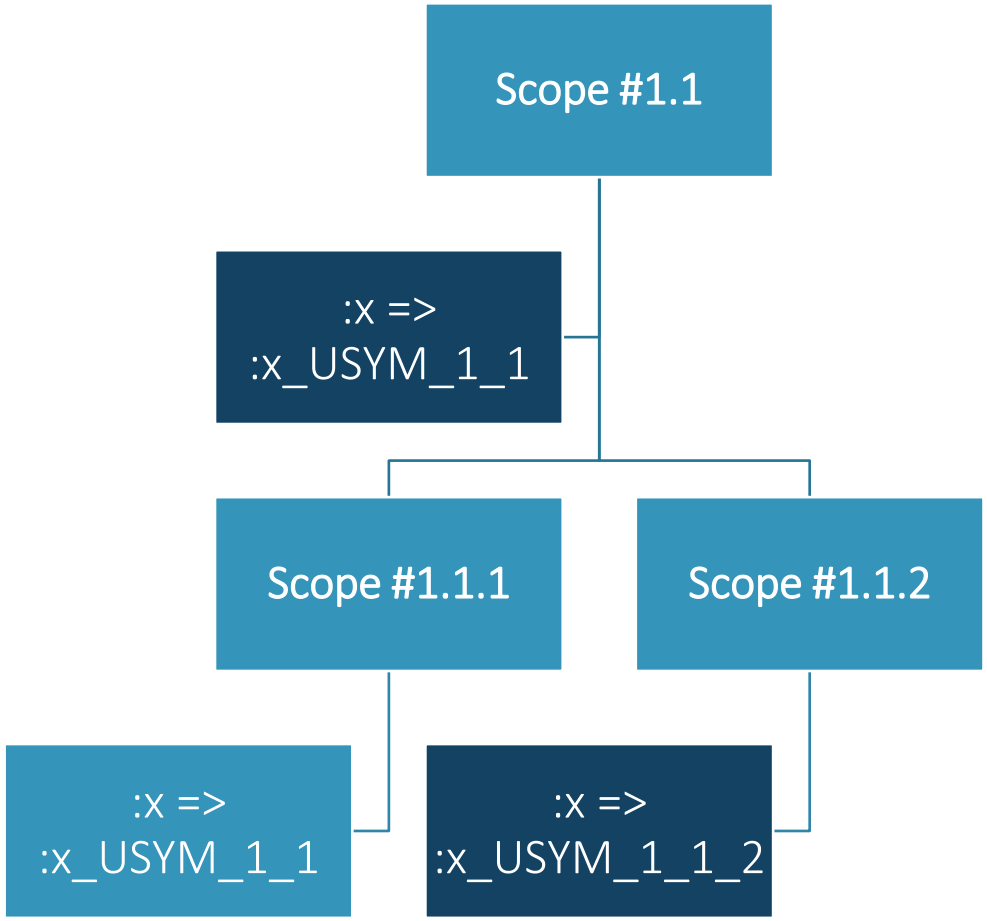
```lua
function usym_example()
    some_global = 2

    global some_global

    some_other_global = 2

    x = 1

    while false
        x = 2
    end

    while false
        local x
        x = 3
    end

    some_global = 3
end
```

```
<Symbol Mappings>:
[Scope #1.1]:
  usym_example => usym_example
  some_other_global => some_other_global_USYM_1_1
  some_global => some_global_USYM_1
  gl_Position => gl_Position
  gl_FragCoord => gl_FragCoord
  x => x_USYM_1_1
  [Scope #1.1.1]:
    x => x_USYM_1_1
  [Scope #1.1.2]:
    x => x_USYM_1_1_2
```

```julia
function usym_example()
    some_global = 2

    global some_global

    some_other_global = 2

    x = 1

    while false
        x = 2
    end

    while false
        local x
        x = 3
    end

    some_global = 3
end
```

```glsl
#version 330 core

// This code was auto-generated by the Julia -> GLSL transpiler

void main() {

    // USYM DECLARATIONS

    int some_other_global_USYM_1_1;
    int x_USYM_1_1_2;
    int x_USYM_1_1;

    // BODY

    some_global_USYM_1 = 2;
    some_other_global_USYM_1_1 = 2;
    x_USYM_1_1 = 1;
    while (false) {
        x_USYM_1_1 = 2;
    }
    while (false) {
        x_USYM_1_1_2 = 3;
    }
    some_global_USYM_1 = 3;
}
```

```julia
function usym_example()
    some_global = 2

    global some_global

    some_other_global = 2

    x = 1

    while false
        x = 2
    end

    while false
        local x
        x = 3
    end

    some_global = 3
end
```



Type Inference

SAST to Typed AST (TAST) transformation

Traverses the tree from bottom to top (post-order)

Infers types for easily deducible nodes (e.g. literals, parameters)

Based on them, it can decide the result type of function calls and operations

It has an interface for pipelines to provide their target language's built-in functions with their signature

Assigns types to the unique symbols as well

Infers return type for the transpiled function

```julia
x = 1.0
y = 2.0

z = x + y
```

```
Expr
  head: Symbol =
  args: Array{Any}((2,))
    1: Symbol z_USYM_1_1
    2: Expr
      head: Symbol call
      args: Array{Any}((3,))
        1: Symbol +
        2: Symbol x_USYM_1_1
        3: Symbol y_USYM_1_1
```

```
[ASTFloat32]
Expr: =
  [ASTVoidSym]
  Symbol: z_USYM_1_1
  [ASTFloat32]
  Expr: call
    [ASTFunction]
    Symbol: +
    [ASTFloat32]
    Symbol: x_USYM_1_1
    [ASTFloat32]
    Symbol: y_USYM_1_1
```

```
mat = mat2x3(0)
t_mat = mat'
```

```
[ASTMat2x3F]
Expr: =
  [ASTVoidSym]
  Symbol: mat_USYM_1_1
  [ASTMat2x3F]
  Expr: call
    [ASTFunction]
    Symbol: mat2x3
    [ASTInt64]
    Int64: 0
[ASTMat3x2F]
Expr: =
  [ASTVoidSym]
  Symbol: t_mat_USYM_1_1
  [ASTMat3x2F]
  Expr: call
    [ASTFunction]
    Symbol: transpose
    [ASTMat2x3F]
    Symbol: mat_USYM_1_1
```

GLSL Transformation

TAST to GLSL IR (intermediate representation) transformation

Traversal order is decided per node

This is a representation meant to follow how GLSL, and its language constructs are structured (but is not based on any official specification)

It has lots of similarities with Julia's AST, but it's overall better suited for code generation (in my opinion)

Utilizes abstract types and subtyping instead of the generic types the Julia AST uses

Decides where to place certain declarations (e.g. interface vs function level)

The final output is basically a 1:1 representation of this stage's output
(except for formatting)

Matrix indexing

```
julia> m = mat3(1:9)
3×3 Matrix{Float32}:
 1.0  4.0  7.0
 2.0  5.0  8.0
 3.0  6.0  9.0
```

```
julia> m[3,2]
6.0f0
```

```
julia> m[6]
6.0f0
```

```
julia> m[:,2]
3-element StaticArr
 4.0
 5.0
 6.0
```

```
@exported struct GLSLMatIndexer <: GLSLASTNode
    target::GLSLASTNode
    column::GLSLASTNode
    row::Union{GLSLASTNode,Nothing}
end
```

```
float six = m[1][2];
```

```
vec3 second_col = m[1];
```

GLSL Code Generation

GLSL IR -> String transformation

Traverses the tree from bottom to top (post-order)

Mostly just provides a textual representation for the GLSL-like AST
(no actual parsing/transforming of the code happens)

Has two modes:

1. Entry point (main function, ignores function-level type information)
2. Helper (other functions in the shader, using param and return type info from TAST)

```
function glsl_cg_traverse(node::GLSLIf, ctx::GLSLCodeGenContext)
    code = "if (" * glsl_cg_traverse(node.condition, ctx) * ") {\n"
    code *= glsl_cg_traverse(node.body, ctx)
    code *= "}"

    for elseif_branch in node.elseif_branches
        code *= " else if (" * glsl_cg_traverse(elseif_branch.condition, ctx) * ") {\n"
        code *= glsl_cg_traverse(elseif_branch.body, ctx)
        code *= "}"
    end

    if !isnothing(node.else_branch)
        code *= " else {\n"
        code *= glsl_cg_traverse(node.else_branch, ctx)
        code *= "}"
    end

    code
end
```

```
function glsl_cg_traverse(node::GLSLSwizzle, ctx::GLSLCodeGenContext)
    glsl_cg_traverse(node.base, ctx) * "." * node.swizzle
end
```

Further Development

Arrays

Structs

Interface Blocks

Unrestricted function transpile

Precompilation workflow improvements

Layout Qualifiers (and const)